PCCCS495 : Advanced Programming(OOP) LAB

Inventory Management System

A Java Console Application using DAO Pattern + File persistence

Student Name: Archisha Basu

Roll Number: 7

Stream: CSE(IOT)

Enrollment Number: 12024052019004

Instructors: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Spring 2026

1. Problem Statement

This project presents a console-based Inventory Management System developed entirely in Java, with the primary objective of demonstrating the practical application of Object-Oriented Programming principles within a structured, multi-layered architecture.
The system is built around the Data Access Object (DAO) design pattern, which enforces a deliberate separation between the user interface, business logic, and data persistence layers. Each layer communicates only with the layer directly below it, ensuring that no component carries responsibilities beyond its defined role. This separation is the central design goal of the project.
The application supports core inventory operations including adding products with auto-generated identifiers, updating product details, adjusting stock levels through restock and reduction operations, and removing products from the system. All data is persisted to a CSV file, ensuring that inventory state is retained across sessions without the overhead of a database.
The project applies five OOP concepts in a deliberate and identifiable manner: encapsulation in the Product model, abstraction through the ProductDAO interface, inheritance in the custom exception hierarchy, polymorphism through constructor injection of the DAO implementation, and exception handling across the service and UI layers. Each concept is used where it is architecturally appropriate rather than for the sake of demonstration alone.

2. System Design

2.1 UML Class Diagram

```
+-------------------+         +----------------------+
|  <<interface>>    |         |       Product        |
|  ProductDAO       |         +----------------------+
+-------------------+         | - productID: int     |
| + save()          |         | - name: String       |
| + findBySKU()     |         | - sku: String        |
| + findAll()       |         | - price: double      |
| + deleteBySKU()   |         | - quantity: int      |
| + updateStock()   |         | - description: String|
| + update()        |         +----------------------+
| + getNextID()     |         | + getters / setters  |
+--------^----------+         | + display()          |
         | implements         +----------------------+
+--------+----------+                   ^
|  FileItemDAO      |                   | uses
+-------------------+       +-----------+---------+
| + all DAO methods |<------| InventoryService     |
| - csvToProduct()  | uses  +---------------------+
| - productToCSV()  |       | - dao: ProductDAO    |
+-------------------+       | + addProduct()       |
                            | + deleteProduct()    |
+-------------------+       | + reduceStock()      |
|  ConsoleMenu      |------>| + restockProduct()   |
+-------------------+ uses  | + updateProduct()    |
| + main()          |       | + getAllProducts()   |
+-------------------+       +---------------------+
```

2.2 Major Classes

Product (com.inventory.model): Core data model. Encapsulates all item attributes: productID, name, SKU, price, quantity, description. Fields are private with getters and setters. Provides toString() for CSV serialisation and display() for console output.

ProductDAO (com.inventory.dao): Interface declaring the data access contract: save, findBySKU, findAll, deleteBySKU, updateStock, update, getNextProductID. Contains no implementation.

FileItemDAO (com.inventory.dao): Concrete implementation of ProductDAO. The only class that touches the filesystem. Uses BufferedReader and BufferedWriter for CSV operations, and a two-phase write strategy for modifications.

InventoryService (com.inventory.service): Business logic layer. Holds a ProductDAO reference. Enforces rules such as duplicate SKU prevention and insufficient stock detection. Throws custom exceptions.

ConsoleMenu (com.inventory.ui): Entry point. Presents menu, collects input, calls InventoryService, and handles exceptions with user-friendly messages. Only class that instantiates FileItemDAO directly.

Exception classes (com.inventory.exception): ItemNotFoundException, InsufficientStockException, DuplicateSKUException. Each extends RuntimeException.

2.3 Package Structure

```
src/com/inventory/
  model/      Product.java
  dao/        ProductDAO.java,  FileItemDAO.java
  service/    InventoryService.java
  exception/  ItemNotFoundException.java
              InsufficientStockException.java
              DuplicateSKUException.java
  ui/         ConsoleMenu.java
```

2.4 Interaction Flow (Add Product example)

- ConsoleMenu collects name, SKU, price, quantity, description from the user.
- ConsoleMenu calls service.addProduct() with the collected fields.
- InventoryService calls dao.findBySKU(). If the SKU already exists, throws DuplicateSKUException.
- InventoryService calls dao.getNextProductID() to generate the next available ID.
- InventoryService creates a Product, sets the ID, and calls dao.save().
- FileItemDAO appends the CSV-serialised product to data/inventory.csv.
- ConsoleMenu catches any exception and prints a friendly error, or prints success.

3. OOP Concepts Demonstrated

3.1 Encapsulation

Where: Product class. All six fields are private, accessed only through getters and setters.

Why: Prevents arbitrary modification of internal state. Setters can enforce validation. SKU has no setter, making it effectively immutable after construction.

```
private int productID;  private String sku;  private double price;

public String getSKU()  { return sku; }
public double getPrice(){ return price; }

public void setPrice(double newprice)      { this.price = newprice; }
public void setQuantity(int newquantity)   { this.quantity = newquantity; }
```

All fields are declared private. External classes read values via getters and modify them only through setters. SKU has no setter, enforcing immutability after creation.

3.2 Abstraction

Where: ProductDAO interface. Declares all data access method signatures with no implementation.

Why: Decouples InventoryService from the storage mechanism. Swapping FileItemDAO for a DatabaseItemDAO requires no changes to the service layer.

```
public interface ProductDAO {
    void save(Product p);
    Product findBySKU(String sku);
    List<Product> findAll();
    void deleteBySKU(String sku);
    void updateStock(String sku, int quantity);
    void update(Product p);
    int getNextProductID();
}
```

The interface is a pure contract. InventoryService stores a ProductDAO reference and calls methods on it without knowing whether the implementation reads from a file, a database, or any other source.

3.3 Inheritance

Where: ItemNotFoundException, InsufficientStockException, and DuplicateSKUException each extend RuntimeException.

Why: Inherits all exception infrastructure (message storage, stack trace, throw/catch support) while providing distinct, semantically meaningful type names for each error condition.

```
public class ItemNotFoundException extends RuntimeException {
    public ItemNotFoundException(String message) { super(message); }
}
public class InsufficientStockException extends RuntimeException {
    public InsufficientStockException(String message) { super(message); }
}
```

Each class passes the message to RuntimeException via super(message). When e.getMessage() is called in a catch block, it retrieves this message. The value is entirely in the named type, making catch blocks precise and readable.

3.4 Polymorphism

Where: InventoryService holds a field of type ProductDAO but is assigned a FileItemDAO instance at runtime via constructor injection in ConsoleMenu.

Why: Allows InventoryService to issue the same method calls regardless of which concrete implementation is behind the interface. The system can target a different storage backend by changing one line in ConsoleMenu.

```
// ConsoleMenu.java — only place FileItemDAO is referenced directly
FileItemDAO dao = new FileItemDAO();
InventoryService service = new InventoryService(dao);

// InventoryService.java — works only with the interface
private ProductDAO dao;
public InventoryService(ProductDAO dao) { this.dao = dao; }
dao.save(p);         // Java routes to FileItemDAO.save() at runtime
dao.findBySKU(sku);  // Java routes to FileItemDAO.findBySKU() at runtime
```

FileItemDAO is passed as a ProductDAO reference. InventoryService never imports FileItemDAO. Java's runtime polymorphism dispatches each interface call to the correct concrete method transparently.

3.5 Exception Handling

Where: InventoryService throws custom exceptions for invalid operations. ConsoleMenu catches them and displays friendly messages.

Why: Separates normal flow from error flow. The service layer detects the problem and throws; the UI layer catches and presents it to the user without exposing stack traces.

```
// InventoryService.java
public boolean reduceStock(String sku, int quantity) {
    Product product = dao.findBySKU(sku);
    if (product == null)
        throw new ItemNotFoundException("Product not found: " + sku);
    if (product.getQuantity() < quantity)
        throw new InsufficientStockException("Stock insufficient");
    dao.updateStock(sku, quantity * -1);
    return true;
}
```

Preconditions are checked before executing the operation. If either check fails, a typed exception is thrown. ConsoleMenu wraps each service call in a try-catch and prints e.getMessage() — the user sees a clear error, not a crash.

3.6 Collections

Where: java.util.List and java.util.ArrayList used in FileItemDAO.findAll() and returned through InventoryService.getAllProducts() to ConsoleMenu.

Why: ArrayList provides a dynamically sized, ordered collection for building up product records line by line from a file. Returning List rather than ArrayList is itself an application of abstraction.

```
public List<Product> findAll() {
    File file = new File("data/inventory.csv");
    if (!file.exists()) return new ArrayList<>();
    List<Product> prodlist = new ArrayList<>();
    // ... read lines, parse, add to prodlist ...
    return prodlist;
}
```

An empty ArrayList is initialised before reading begins. Each non-null product parsed from a CSV line is added. If the file does not exist, an empty list is returned safely, preventing a crash on first run.

4. Implementation Highlights

4.1 Two-Phase File Write with Atomic Move

When modifying or deleting a CSV record, the file cannot be edited in place. The system writes all lines to a temporary file with the required change applied, then uses Files.move() with REPLACE_EXISTING to atomically swap the files. Critically, Files.move() is placed in a separate try block after the reader and writer are closed — on Windows, a file cannot be replaced while any process holds it open, causing a FileSystemException if placed inside the same try-with-resources.

```
try (BufferedReader reader = new BufferedReader(new FileReader(file));
     BufferedWriter writer = new BufferedWriter(new FileWriter("data/temp.csv"))) {
    // write modified lines to temp.csv
} catch (IOException e) { e.printStackTrace(); return; }

// Second block — handles are now fully closed
try {
    Files.move(Paths.get("data/temp.csv"),
        Paths.get("data/inventory.csv"), StandardCopyOption.REPLACE_EXISTING);
} catch (IOException e) { e.printStackTrace(); }
```

4.2 Auto-Generated Product ID

The system generates product IDs automatically by scanning all existing records and returning the highest ID plus one. This removes the burden from the user and prevents duplicate IDs.

```
public int getNextProductID() {
    List<Product> all = findAll();
    int maxID = 0;
    for (Product p : all) {
        if (p.getProductID() > maxID) maxID = p.getProductID();
    }
    return maxID + 1;  // returns 1 if inventory is empty
}
```

4.3 Graceful First-Run Handling

On the first run, data/inventory.csv does not exist. Every read method checks for file existence before attempting to open it, returning a safe default (null or empty list) instead of throwing a FileNotFoundException. The static initialiser ensures the data/ directory is created on class load.

```
static { new File("data").mkdirs(); }

public List<Product> findAll() {
    File file = new File("data/inventory.csv");
    if (!file.exists()) return new ArrayList<>();
    // ... proceed with reading
}
```

4.4 Duplicate SKU Prevention

Before saving a new product, InventoryService checks whether the SKU exists. If it does, DuplicateSKUException is thrown and no file operation is performed, preserving inventory integrity at the point of insertion.

```
public boolean addProduct(String name, String sku, double price,
        int quantity, String description) {
    if (dao.findBySKU(sku) != null)
        throw new DuplicateSKUException("SKU already exists: " + sku);
    int newID = dao.getNextProductID();
    Product p = new Product(name, sku, price, quantity, description);
    p.setProductID(newID);
    dao.save(p);
    return true;
}
```

5. Testing and Error Handling

5.1 Test Cases

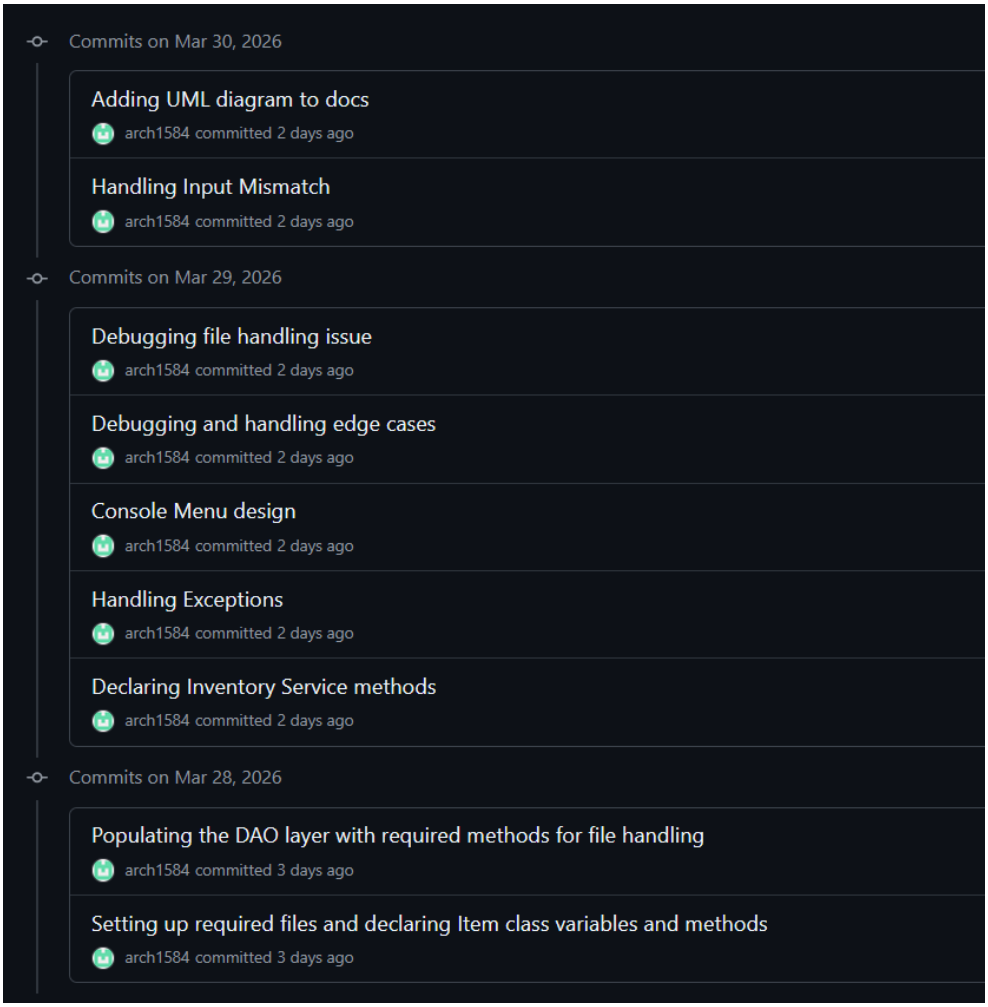
- Add a new product: product appears in View All; CSV entry written with correct auto-generated ID.
- Add duplicate SKU: DuplicateSKUException thrown; no duplicate written to file.
- View all on empty inventory: message displayed; no crash.
- Find by valid SKU: correct product details displayed.
- Find by non-existent SKU: friendly error message; no crash.
- Reduce stock by valid quantity: updated quantity displayed immediately after.
- Reduce stock below available: InsufficientStockException thrown; stock unchanged.
- Restock a product: quantity increases correctly; confirmed by subsequent Find.
- Update price and description: changes reflected in View All.
- Update non-existent SKU: ItemNotFoundException thrown before prompting for new values.
- Delete a product: product absent from subsequent View All.
- Enter letters at menu: invalid input message; menu re-shown without crash.
- Restart application: all data loads correctly from CSV; persistence confirmed.

5.2 Edge Cases and Failure Scenarios

- File does not exist on first run: all read methods return safe defaults; mkdirs() creates data/ directory.
- Empty lines in CSV: csvToProduct() returns null for blank lines; null check skips them.
- Malformed CSV line: NumberFormatException caught inside csvToProduct(); line is skipped gracefully.
- Reduce stock to exactly zero: permitted; quantity becomes zero, product record remains.
- Restock with negative input: absolute value taken; treated as positive addition.
- FileSystemException on Windows during file move: resolved by separating Files.move() into a second try block after file handles are closed.

6. Git Workflow

6.1 Commit History Screenshot



6.2 Development Progression

1. Different operational layers of src/ created; dao/, model/, service/, ui/, exceptions/. And also added Product model: Product.java with private fields, constructor, getters, setters, toString(), display().
2. Added ProductDAO interface: all method signatures declared and implemented FileItemDAO: CSV read/write methods implemented.
3. Added InventoryService: business logic layer with all operations and exception throwing.
4. Added custom exceptions: ItemNotFoundException, InsufficientStockException, DuplicateSKUException.
5. Added ConsoleMenu: main menu loop with all cases and try-catch blocks.
6. Bug fix: two-phase file write: FileSystemException on Windows resolved by separating Files.move() into a second try block.
7. Bug fix: first-run handling: file existence checks added to all read methods.
8. Bug fix: auto-generated ID: user input for product ID removed; getNextProductID() implemented. And also implemented readInt() and readDouble() methods to manage input mismatch scenarios.
9. Final cleanup: duplicate SKU check added; display() corrected to void return type and UML design added in slides/ folder.

7. Conclusion and Future Scope

7.1 Conclusion

This project demonstrates a well-structured Inventory Management System implemented entirely in Java. The system achieves clean separation of concerns across four distinct layers: model, data access, service, and user interface. The DAO pattern is applied intentionally, and each core OOP principle like encapsulation, abstraction, inheritance, polymorphism, and exception handling is used deliberately and identifiable at a specific location in the codebase.

The system handles real failure scenarios encountered during development, including Windows file-locking issues and first-run conditions. These were resolved through design decisions rather than workarounds. The project prioritises design quality and defensibility over feature count, consistent with the evaluation criteria.

7.2 Future Scope

- Database persistence: a DatabaseItemDAO implementing ProductDAO could replace FileItemDAO with no changes to any other layer.
- Low-stock alert thread: a background thread scanning inventory periodically and logging warnings when quantity falls below a threshold.
- Multiple warehouse support: additional DAO implementations targeting different CSV files, selectable at startup.
- Graphical interface: the strict separation of ConsoleMenu from InventoryService means the UI layer could be replaced with Swing or JavaFX without touching any other layer.
- Search and filter: methods such as findByName() or findByPriceRange() added to ProductDAO and implemented in FileItemDAO.

Appendix

A. Initial Project Proposal

Project Title: Inventory Management System using DAO Pattern + File Persistence

Name: Archisha Basu **Stream: CSEIOT** **Roll: 7**

Problem Statement: Small businesses or shop owners often manage inventory manually using spreadsheets or paper records. This approach makes it difficult to track stock levels accurately, handle updates efficiently, and maintain consistent records over time. The proposed Inventory Management System addresses these issues by separating data access logic from business logic using the Data Access Object (DAO) pattern. The system will manage inventory items through a structured object-oriented design while storing data persistently in files which ensures that inventory data is retained between program executions without requiring a full database system.

Target User: Small retail shop owners managing daily stock or warehouse supervisors tracking inventory levels.

Core Features:

- Add new inventory items (ID, name, quantity, price, category)
- Update existing product details
- Delete products from inventory
- Search items by ID or name
- Automatic stock level updates after transactions
- Low-stock alerts for items below a threshold
- File-based persistent storage
- Input validation and robust exception handling

OOP Concepts to be Used:

- **Abstraction**: Abstract DAO interface defining standard data access operations (create, read, update, delete)
- **Encapsulation**: Inventory item attributes managed through getter and setter methods
- **Inheritance**: DAO implementation classes extending the DAO interface
- **Polymorphism**: Different DAO implementations (like FileInventoryDAO) implementing the same interface
- **Exception Handling**: Handling invalid input, file read/write errors, and duplicate product IDs
- **Collections**: Use of lists or maps to store and manipulate inventory items in memory

Brief Architecture Description: The system follows a layered object-oriented architecture that separates business logic from data storage. The InventoryItem class represents products with attributes such as ID, name, quantity, price, and category. The InventoryDAO interface defines standard CRUD operations for inventory management which is implemented by the FileInventoryDAO class. InventoryManager handles business logic such as stock updates and search operations.

Inventory Item Model → DAO Interface → File DAO Implementation → Inventory Manager → File Persistence Layer